

Relatório do Trabalho Final de Compiladores

Guilherme Brizzi¹

¹Curso de Ciência da Computação –
Universidade Federal de Santa Maria (UFSM)

gbrizzi@inf.ufsm.br

1. Introdução

O Home Assistant é um software de código aberto líder em automação residencial, permitindo a integração de milhares de dispositivos. Neste sistema, a especificação das automações é representada nativamente por arquivos YAML, que definem os gatilhos, condições e ações de cada rotina. O usuário pode editá-las por meio de uma Interface Gráfica (GUI), caso queira abstrair essa complexidade de nível mais baixo. Porém, para acessar todo o poder do Home Assistant e criar automações altamente dinâmicas e baseadas em templates complexos, editar diretamente os códigos é essencial.

Contudo, a especificação utilizada no YAML do Home Assistant é frequentemente caótica, verbosa e complexa. A falta de uma estrutura clara de controle de fluxo pode gerar arquivos enormes e difíceis de manter. Dessa forma, propor uma linguagem de especificação mais simples, programática e legível seria extremamente útil para os usuários avançados.

Esse trabalho propõe e implementa a linguagem *Domus*: uma linguagem de especificação enxuta para o Home Assistant que transpila diretamente para códigos YAML compatíveis. A linguagem foi desenhada do zero, abstraindo as idiossincrasias do YAML e oferecendo uma sintaxe inspirada em linguagens modernas.

2. Construção da Linguagem

A linguagem Domus buscou ser simples, intuitiva e legível. Dessa forma, sua filosofia de design se baseia em quatro princípios fundamentais:

- **Deve haver uma maneira de fazer as coisas:** O Home Assistant frequentemente oferece múltiplas formas (como o comando `choose` ou condicional de `if/then`) para o mesmo controle de fluxo. Domus elimina essas redundâncias, adotando construções únicas e bem definidas.
- **Menos símbolos, por favor!:** Domus limita o uso de símbolos não textuais e aninhamentos profundos (como `{`, `!`, `&&`, `-`), tornando o código mais natural e menos propenso a erros de sintaxe e indentação.
- **Blocos essenciais devem ser claros:** Domus renomeia e reestrutura os blocos principais do Home Assistant. Em vez dos substantivos *triggers*, *conditions* e *actions*, a linguagem adota verbos de ação que indicam o comportamento de cada etapa: *listen* (escutar eventos), *when* (quando a condição for atendida) e *do* (fazer a ação).
- **English, motherf*****, do you speak it?:** Domus é escrita inteiramente em inglês, mantendo a interoperabilidade e sendo acessível para um público internacional, evitando a mistura confusa de inglês com português.

2.1. Gramática Livre de Contexto

A gramática livre de contexto (GLC) da linguagem Domus, que dita todas as regras sintáticas e derivações possíveis a partir do símbolo inicial, está disponível detalhadamente no arquivo `GLC.txt` anexo ao projeto.

2.2. “Hello World” de Domus

O seguinte excerto de código representa um esqueleto típico de uma automação escrita em Domus, demonstrando sua clareza em relação a um YAML tradicional.

```
1 create 'my first automation':  
2   id auto  
3   description 'some description here'  
4   mode single  
5  
6   listen [  
7   ]  
8  
9   when [  
10  ]  
11  
12  do [  
13  ]
```

Listing 1. Exemplo básico de automação em Domus

2.3. Mais informações

Em Domus, convencionou-se utilizar a extensão `.domus` para os arquivos fonte. Recomenda-se fortemente o uso de indentação, conforme os exemplos fornecidos, para facilitar a legibilidade humana. No entanto, do ponto de vista do compilador, a indentação não é estritamente obrigatória (a linguagem é *free-form*, ignorando espaços em branco extras).

3. Construção do Transpilador

O projeto foi dividido nas etapas clássicas de um compilador/transpilador de *front-end*, recebendo o código fonte em Domus, construindo a representação interna e emitindo o YAML alvo.

3.1. Analisador Léxico (*Lexer*)

Para a construção do analisador léxico, utilizou-se a ferramenta geradora de lexers Flex. Com ela, foram definidos os tokens utilizados na linguagem por meio de expressões regulares. O lexer é responsável por ignorar espaços e comentários, além de identificar palavras-reservadas (`listen`, `when`, `do`, etc.), identificadores, strings e numerais. Sua implementação detalhada encontra-se no arquivo `src/lexer.l`.

3.2. Analisador Sintático (*Parser*)

O analisador sintático foi desenvolvido utilizando a ferramenta Bison. Nela, foi transcrita a GLC e associadas ações semânticas específicas (em código C) a cada regra gramatical para a construção progressiva da Árvore Sintática Abstrata (AST). A implementação está disponível no arquivo `src/parser.yy`.

O Bison, por padrão, adota uma estratégia *Bottom-Up*, mais especificamente o algoritmo LALR(1), uma variação eficiente e rápida da abordagem LR. Essa escolha abstrata provida pela ferramenta facilitou imensamente a programação do parser, garantindo o tratamento determinístico da gramática sem a necessidade de passos complicados de refatoração, como fatoração à esquerda ou eliminação de recursão à esquerda, comumente exigidos por parsers *Top-Down* (LL).

3.3. Analisador Semântico

A análise semântica realiza travessias sobre a AST construída pelo *parser* para garantir que o significado do código é válido. O analisador garante que as relações geradas façam sentido no contexto do Home Assistant – como a verificação de tipagem correta, argumentos obrigatórios fornecidos, ações e gatilhos permitidos, e a estruturação lógica das regras. A implementação estrutural da AST e a verificação semântica encontram-se nos arquivos `src/ast.c`, `src/semantic.c`, `src/symtab.c`.

3.4. Tratamento de Erros

Como requisito de robustez, o compilador adota a estratégia de **Modo Pânico** (*Panic Mode*) para a recuperação de erros sintáticos. Assim, ao esbarrar em um token inesperado (um erro sintático), o analisador não aborta imediatamente; ao contrário, ele emite uma mensagem de erro indicando a linha problemática e tenta se ressincronizar. Essa ressincronização é feita descartando os tokens subsequentes da entrada até encontrar um token de sincronização seguro – que, no caso de Domus, foi definido primariamente como o fechamento de parênteses (`)`). Isso permite que o compilador encontre múltiplos erros em uma única execução.

3.5. Geração de Código

Finalmente, caso o programa fonte passe sem erros por todas as etapas pregressas, o código YAML alvo é emitido. O módulo `src/yaml.c` contém o núcleo da geração de código, operando através de uma travessia final na AST validada.

Como o YAML, especialmente na formatação esperada pelo Home Assistant, possui bastante complexidade, é verboso e relativamente irregular em algumas de suas definições (uso de listas *versus* dicionários), o módulo gerador requereu a programação de inúmeros casos especiais. A emissão deve gerenciar meticulosamente os níveis de indentação, o uso de hífen para listas e o escape correto de aspas em templates complexos, para lidar corretamente com a compilação completa das automações descritas na linguagem Domus.

4. Demonstrações, Exemplos e Execução

O compilador foi desenvolvido para ser utilizado de maneira simples via linha de comando. Recomenda-se o uso em ambiente Linux (ou WSL em Windows) e é necessário possuir instalados: `flex`, `bison`, `gcc` e `make`.

Para construir o executável do transpilador, basta executar na raiz do projeto o comando:

```
1 make
```

A pasta `examples` na raiz do projeto concentra todos os arquivos de demonstração, estruturada em três subpastas organizacionais:

- `domus/`: Contém os códigos fonte em Domus (extensão `.domus`).
- `yaml/`: Contém os exemplos originais de referência em YAML, com base na documentação do Home Assistant.
- `output/`: Diretório de destino que recebe os artefatos finais gerados pelo compilador Domus.

Ao executar o comando `make run`, o sistema de *build* processa automaticamente todos os exemplos contidos em `examples/domus/ex*.domus`, gerando seus respectivos equivalentes YAML em `examples/output/`.

Para testar o compilador individualmente ou com scripts customizados, o comando segue a notação padrão de *standard input/output*:

```
1 ./domus-transpiler < input.domus > output.yaml
```

A fim de validar as rotinas de tratamento de erro, foram disponibilizados três arquivos adicionais para testes rigorosos: `examples/domus/broken_*`. Esses arquivos contêm, propositalmente, construções que disparam falhas nas etapas léxica, sintática e semântica, ativando as rotinas de log e recuperação (*Panic Mode*) do compilador.

5. Conclusão

O projeto atingiu seu objetivo primordial: abstrair a complexidade de definição de automações em YAML e prover uma sintaxe mais natural através da linguagem Domus. A estrutura desenvolvida (Lexer, Parser, AST e Code Generator) demonstrou robustez suficiente para validar semanticamente o fluxo lógico e traduzi-lo de maneira correta.

Como trabalhos futuros, a extensão da linguagem para suportar *blueprints* (templates genéricos de automação) e a possível integração direta com a API do Home Assistant poderiam oferecer um ambiente unificado, sem a necessidade da etapa manual de importação dos arquivos YAML transpilados para a plataforma.